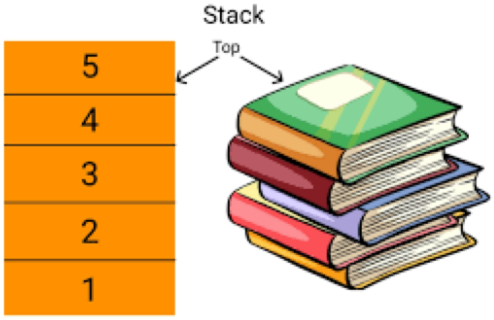
STACK

DAY 49
28/08/25



STACK IS A Type of DATA STRUCTURE

A linear Data structure that follows the Principle LIFO (Last In, First Out)

Real Life Analogies of stack

① stack of plates In a Cafeteria
(push) if you add a new plate on the top
(POP) you remove that plate first

② Undo / Redo in the text Editors
- Every Action is stored In a stack.
Undo Pops the last Action

Operation on Stack

① push (x) → Insert a Element X on top
② POP → Remove top element
③ Peek() / Top() → get the top Element without removing
④ IsEmpty() → check if stack is Empty.
⑤ Isfull() → (for fixed size stacks)

Implimentation of Stack

① Using arrays (fixed size, But faster access)
② Using LinkedList (dynamic size)
③ Java provides Built-in stack class in java.util.Stack

One of the Best Example of stack we can think of is CALLSTACK (function call stack)

When the program is executed if there are any function calls (more likely multiple calls) on each function call a call stack is created

We can Observe through Debugging

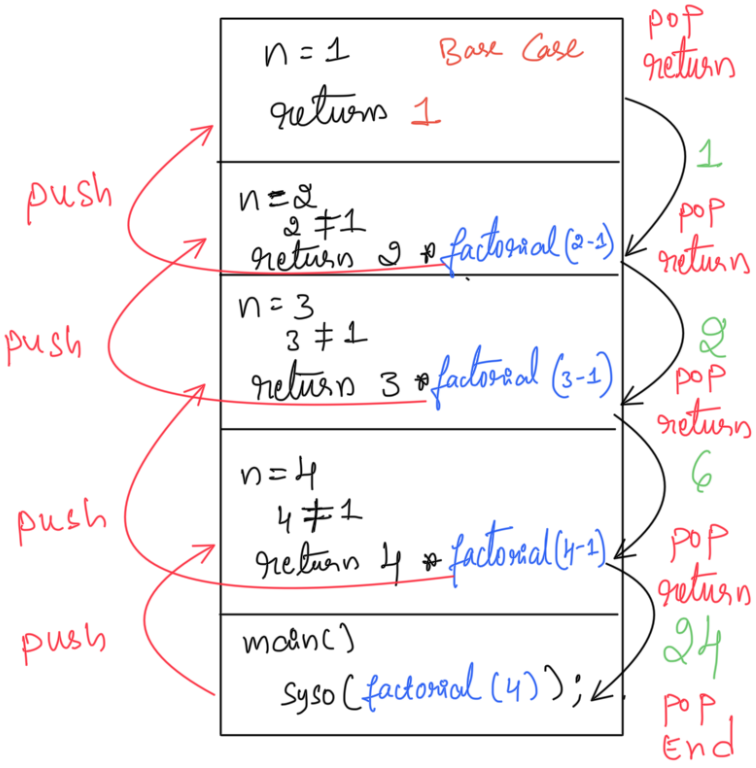
A call stack is a special type of stack Data Structure used by programming language runtime to keep track of function calls.

- When a function is called, its details are Pushed onto stack.
- When a function, its details are Popped from the stack.

What does the call stack store | stack frame store

- Function Name
- Parameters
- Local Variables
- Runtime address (where to continue after the function ends)

The Recursion Heavily relies on call stack, let us understand with Example

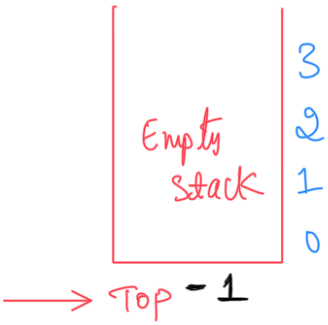


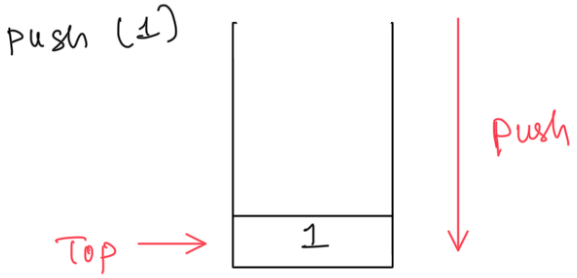

```java
static int factorial(int n) {
    if (n == 1) return 1;
    return n * factorial(n - 1);
}

public static void main(String[] args) {
    System.out.println(factorial(4));
}
```

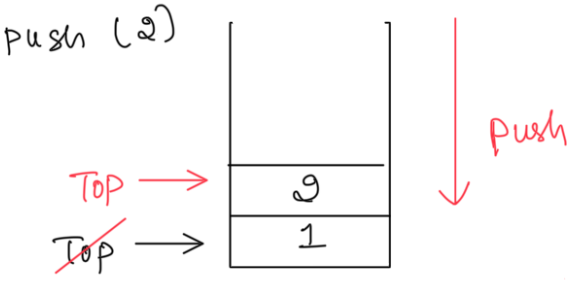
```
factorial(4) → waits for factorial(3)
factorial(3) → waits for factorial(2)
factorial(2) → waits for factorial(1)
factorial(1) returns 1
factorial(2) returns 2
factorial(3) returns 6
factorial(4) returns 24
```

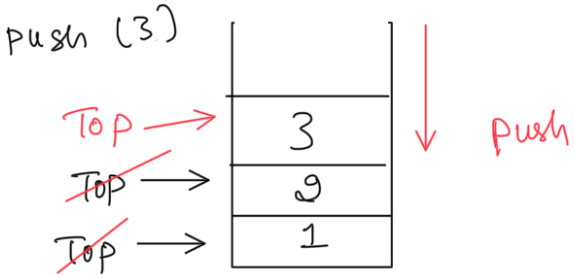
NOTE : Without a Proper Base Case, recursion Can Cause a Stack Over Flow Error Because Java Call Stack Have limited Memory.

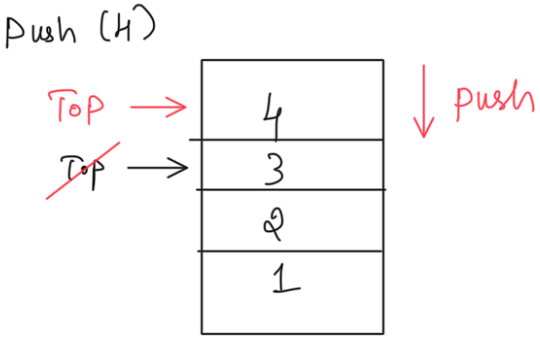


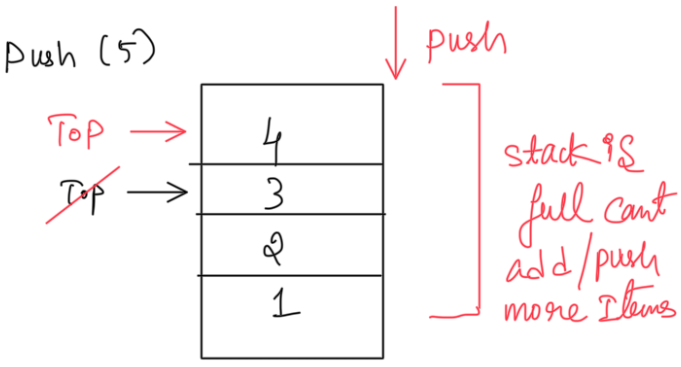


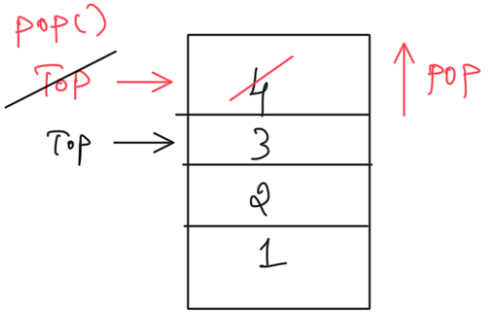
push pop operation on stack Have Capacity or Size 4

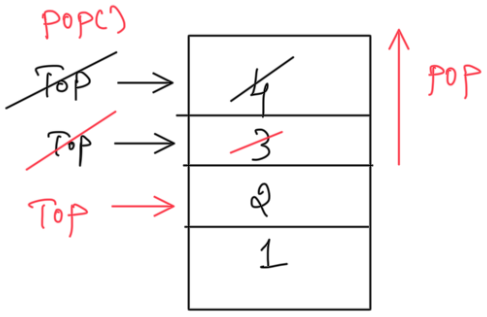


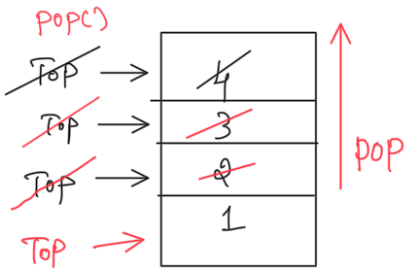


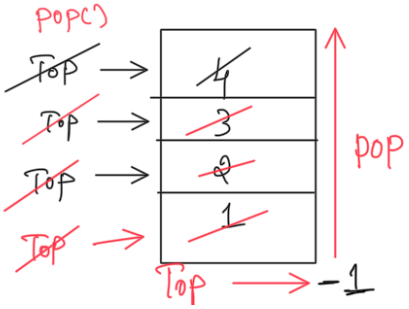


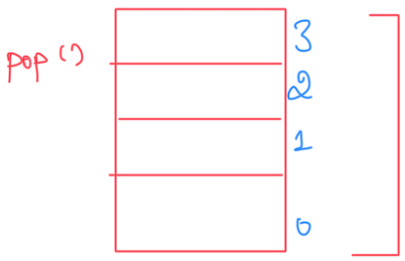












Stack is Empty Further no Elements to pop

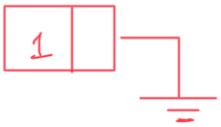
Initially Top is pointing to -1 in Array Based Stack Implementation and Null in case of Linked List Based Stack Implementation.

Top is updated on Each push() pop() operations to keep track of the Top most Element in Stack.

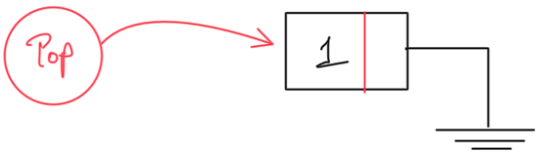
understanding stack with the help of Linked List

Initially top = Null stack is Empty

push (1) Create a Node

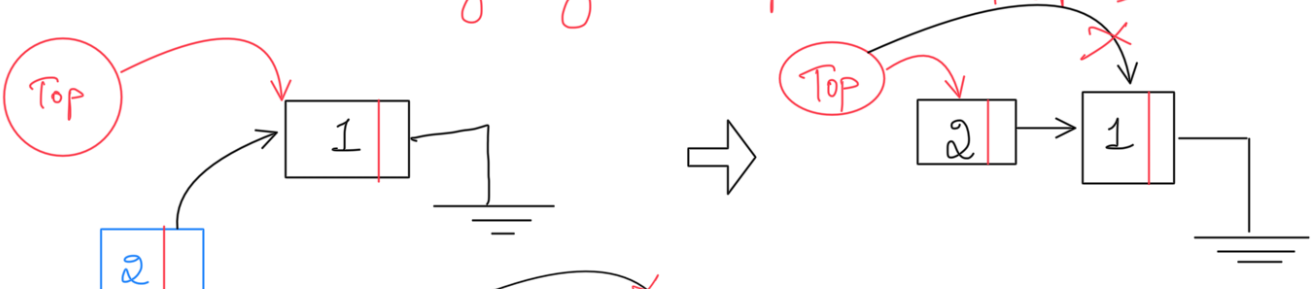


point top to the newly created Node

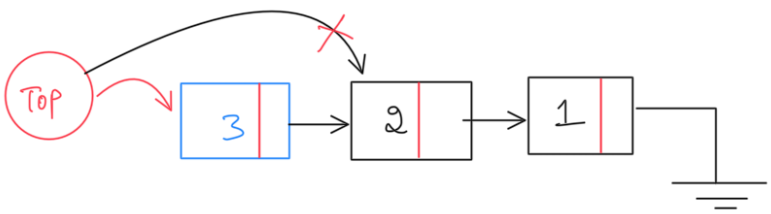


push (2) Create a Node

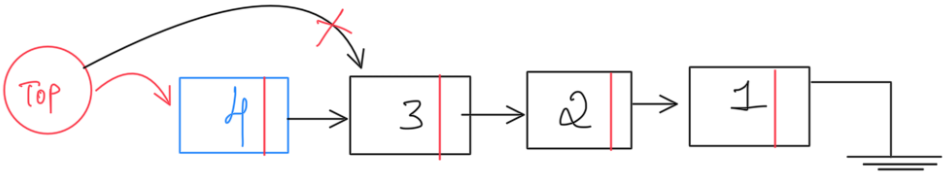
point newly created Nod Next to Head/Top of linked list and update Head/Tail point it to New Node (Basically For form Insert At Beginning and update Head/Top)



push (3)

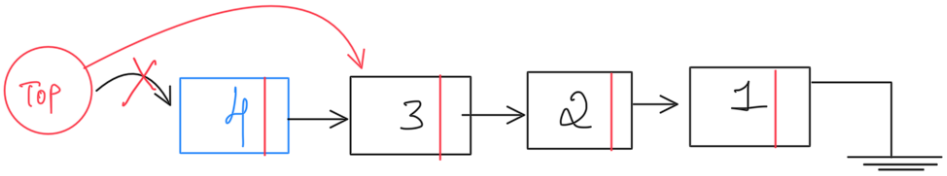


push (4)



If Top is pointing to Null then Stack is Empty

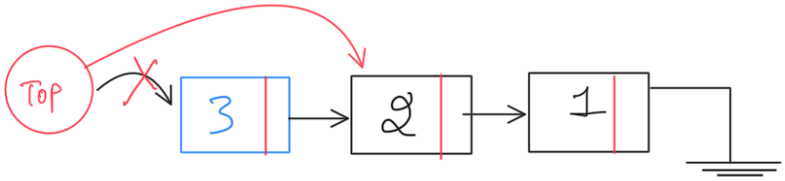
pop



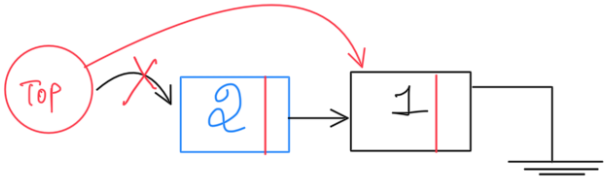
if we want to perform pop operation simply Point Top/Head to top.next. Old top pointing Node was deleted By garbage Collector.

In Simple Words perform deleteAtFront.

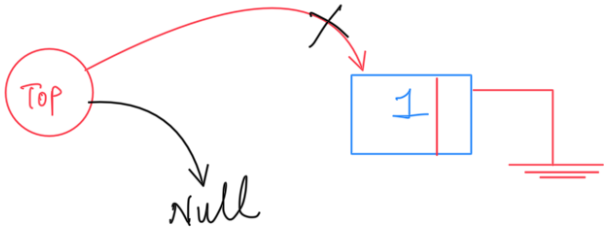
pop



pop



pop



pop

"Null" Can't perform pop Bcz the stack is Already Empty.

Tomorrow I will Implement stack In Java using Both Array and Linked List.

See you Tomorrow